OPENPLANR · v1.7 · PIPELINE v0.9.1 · SKILL v1.6

Brief in.
Code out.

An agentic planning layer
for AI coding agents.

Backlog · Spec · Decompose · Estimate · Ship · Sync

End to end. In your repo. On any runtime.

OpenPlanr
openplanr.dev

Asem Abdo
asemdevs

THE FLOW

From one-line idea to shipped code.

01 · CAPTURE
Backlog + Spec
`planr backlog · planr spec create + shape`

02 · DECOMPOSE
Stories + Tasks (with rationale)
`/planr-pipeline:plan {slug}`

03 · ESTIMATE
Token / dollar / time gate
`COST ESTIMATE before DEV dispatch`

04 · SHIP
8 agents, manifest-bound
`/planr-pipeline:ship {slug}`

05 · SYNC
Linear · GitHub Issues
`planr sync · bidirectional · three-way merge`

STEP 01 · CAPTURE

From rough idea to real spec.

Drop ideas into the backlog. When ready, shape them into specs through a guided 4-question flow. ADR-aware — your binding architectural decisions inject into the context automatically.

```
~/voult · zsh

$ planr backlog add "AI-augmented support inbox"
↳ BL-014 created

$ planr spec create "AI support inbox" --slug support-inbox
↳ SPEC-007-support-inbox/ scaffolded

$ planr spec shape SPEC-007
  Q1: Who is this for?
  Q2: What does success look like?
  Q3: What constraints apply?
  Q4: What's out of scope?
↳ SPEC-007 ready · ADRs scanned · ready to decompose
```

STEP 02 · DECOMPOSE

Tasks that explain themselves.

The specification-agent reads your spec and breaks it into Stories + Tasks. Every task carries a rationale — 1-3 sentences on why this task exists and which files it touches. QA reads it back to catch drift.

.planr/specs/SPEC-007/tasks/T-003-drizzle-ticket-schema.md

```yaml
---
id: "T-003"
title: "Add Drizzle migration for ticket schema"
storyId: "US-002"
status: "pending"
agent: "backend-agent"
rationale: >
  The ticket schema blocks all inbox UI work — landing it
  first unblocks US-003, US-004, US-005. Drizzle (not Prisma)
  per ADR-002. Migration must be reversible.
files_create: [src/db/migrations/0003_ticket.sql]
files_modify: [src/db/schema.ts]
files_preserve: [src/db/migrations/0001_*, 0002_*]
---
```

STEP 03 · ASSIGN

Eight specialists. One contract.

Each agent has its own model tier and tool restrictions — written in the plugin manifest, enforced at the tool layer. The frontend agent literally cannot write backend files.

db-agent — Sonnet 4.6
READ-ONLY DB introspection.
SQL + Mongo.

designer-agent — Sonnet 4.6
PNG mockups → design-spec.md.
Feature-scoped.

specification-agent — Sonnet 4.6
Spec → Stories + Tasks
with rationale.

frontend-agent — Opus 4.7
UI codegen.
R6 correction loop.

backend-agent — Opus 4.7
Backend codegen.
R6 correction loop.

qa-agent — Sonnet 4.6
DoD gate.
Runs build + tests.

devops-agent — Sonnet 4.6
docker-compose + CI.
Non-deploy enforced.

doc-gen-agent — Sonnet 4.6
Generates Docs/feat-*/.
Keeps docs honest.

STEP 04 · ESTIMATE

Know the bill before it ships.

Every /ship prints a non-binding COST ESTIMATE before DEV dispatch — token range, dollar cost, time. Explicit "proceed" gate. Use --yes for CI.

```
/planr-pipeline:ship support-inbox  ·  Phase B

COST ESTIMATE — support-inbox
  Dispatch:  5 tasks   Runtime: claude-code → multi-task

Task       Title                              Modify      Agent
T-001      Drizzle ticket schema              0           backend
T-002      Inbox list page                    1           frontend
T-003      Ticket detail drawer               1           frontend
T-004      Classify endpoint                  0           backend
T-005      QA + smoke run                     0           qa

  Est. tokens:      ~95k–140k input  /  ~14k–22k output
  Est. cost:        $1.40–$2.20
  Est. time:        12–19 min
  Reply "proceed" — or --task T-NNN to narrow
```

STEP 05 · SHIP

Boundaries enforced. Memory injected.

Each task dispatches to its own isolated subagent on Claude Code — no cumulative-context bias. Project memory (decisions, traps, corrections) auto-injects into every dispatch. Your AI learns from yesterday.

MANIFEST BOUNDARIES

agent.tools[]

frontend	src/components/**	ALLOW
frontend	src/api/**	BLOCK
backend	src/api/**	ALLOW
backend	src/components/**	BLOCK
devops	.github/**	ALLOW
devops	src/**	BLOCK

PROJECT MEMORY

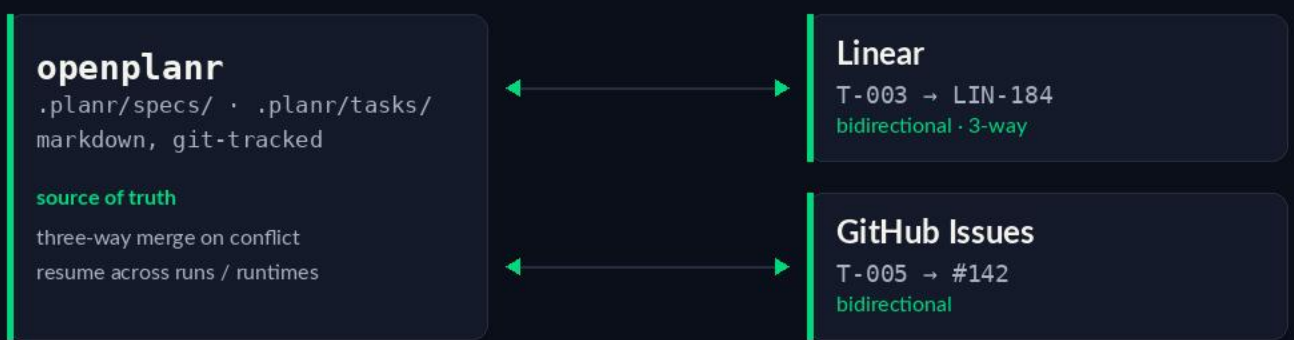
.planr/memory.md

```
## Decisions
- Drizzle ORM, not Prisma.
  Better TS inference.

## Traps
- Prisma createMany doesn't
  support nested relations on
  PG — use $transaction.

## Corrections
- 2026-05-08: never mock
  the DB in integration
  tests. Use real container.
```

STEP 06 · SYNC

Plans live where your team works.

Bidirectional Linear sync with three-way merge. GitHub Issues too. Your plan stays in markdown in your repo — git-blameable, code-reviewable — and stays in sync with whatever your team actually uses.



TRY IT

Two commands. Whole stack.

INSTALL THE CLI

```
$ npm install -g openplanr
```

ADD THE MARKETPLACE IN CLAUDE CODE

```
$ /plugin marketplace add openplanr/marketplace
```

OPEN SOURCE · MIT

github.com/openplanr · monorepo — CLI · pipeline · skill · marketplace

openplanr.dev · docs, protocol spec, runtime compatibility matrix